

TECH

FORMS / VALIDATION

FORMS / VALIDATION

FORMS / VALIDATION

20 ВОПРОСОВ 7 ДНЕЙ КОНСПЕКТ ЛЕКЦИЙ

ИСТОЧНИК react-hook-form / zod / общая практика

20 ВОПРОСОВ 7 ДНЕЙ

ДЕНЬ 1 СРЕДНИЙ	СТРУКТУРА D Q1 · Q2 · Q3 · Q4	CONTROLLED vs UNCONTROLLED	Как разрезать сложную форму и какой подход выбрать.
ДЕНЬ 2 СРЕДНИЙ	ВАЛИДАЦИЯ ИЗАЦИЯ Q5 · Q6 · Q7 · Q8	CLIENT / SERVER / СИНХРОН	Где валидировать и как показывать ошибки.
ДЕНЬ 3 СРЕДНИЙ	TOUCHED / DIRTY STALE Q9 · Q10 · Q11	ASYNC VALIDATION	Управление состоянием полей и async-проверками.
ДЕНЬ 4 ТЯЖЁЛЫЙ	MULTI-STEP Q12 · Q13 · Q14	DRAFTS AUTOSAVE	Длинные wizards и сохранение состояния.
ДЕНЬ 5 СРЕДНИЙ	OPTIMISTIC SUBMIT DISABLED Q15 · Q16 · Q17	DOUBLE SUBMIT	Защита от дублей и UX отправки.
ДЕНЬ 6 ТЯЖЁЛЫЙ	FILE UPLOAD Q18 · Q19	LARGE FORM PERFORMANCE	Файлы и формы с сотней полей.
ДЕНЬ 7 СРЕДНИЙ	ТЕСТИРОВАНИЕ Q20	COMPLEX FORM	Как покрыть форму тестами.

РИТМ 30-60 МИН ТЕОРИЯ 15-30 МИН КОД В КОНСОЛИ 15 МИН ОТВЕТЫ ВСЛУХ

ДЕНЬ 7 — БЕЗ НОВОЙ ТЕОРИИ ТОЛЬКО ПРОГОН ПО 20 ВОПРОСАМ

ДЕНЬ 1 НАГРУЗКА СРЕДНИЙ 4 ВОПРОСА

СТРУКТУРА CONTROLLED vs UNCONTROLLED

ЧТО УЧИМ В ЭТОТ ДЕНЬ

Как разрезать сложную форму и какой подход выбрать.

ВОПРОСЫ ДНЯ

- Q01 Как проектировать сложную форму?
- Q02 Controlled или uncontrolled inputs?
- Q03 Когда использовать React Hook Form?
- Q04 Когда форма без библиотеки лучше?

Как проектировать сложную форму?

ОПИСАНИЕ

Сложная форма – это валидация, conditional fields, multi-step, async-проверки, optimistic-сохранение. Архитектурно: схема валидации отдельно (Zod), контейнер формы отдельно, поля как мелкие компоненты с одним ответственным API.

КАК РАБОТАЕТ

- 01 Schema-first: Zod-схема → тип через `z.infer`.
- 02 Контейнер: `react-hook-form useForm + zodResolver`.
- 03 Поля как маленькие `<FormField name="..." />`.
- 04 Conditional fields через `watch()`.
- 05 Submit handler возвращает Promise – `onSubmit` ждёт.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Регистрация / профиль.
- 02 Конструкторы продукта.
- 03 Multi-step wizards (см. Q12).

ПОДВОДНЫЕ КАМНИ

- 01 Все поля `controlled` – лагает при много полях.
- 02 Без `runtime`-валидации – баги типов.
- 03 Слишком много `conditional` – нечитаемо.
- 04 Validation на `submit`, а не на `blur` – UX страдает.

КОД

// Скелет

```
const Schema = z.object({ email: z.string().email() });
type V = z.infer<typeof Schema>;
const { register, handleSubmit } = useForm<V>({ resolver: zodResolver(Schema) });
```

МОГУТ ДОСПРОСИТЬ

- 01 Как делать conditional fields?
- 02 Где жить адаптеры `FormValues` → API DTO?

ПОЛНАЯ СТАТЬЯ

<https://react-hook-form.com/>

ВОПРОС Q02

Controlled или uncontrolled inputs?

ОПИСАНИЕ

Controlled: значение в state, каждое нажатие – setState и ре-рендер. Uncontrolled: значение в DOM, читаем через ref. Controlled – для динамики (форматирование, валидация в реальном времени), uncontrolled – для performance и больших форм. react-hook-form работает на uncontrolled.

КАК РАБОТАЕТ

- 01 Controlled: `<input value={v} onChange={...} />`.
- 02 Uncontrolled: `<input defaultValue={v} ref={r} />`.
- 03 Hybrid: defaultValue + onBlur валидация.
- 04 react-hook-form: uncontrolled by default + Controller для controlled.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Live-валидация / маски – controlled.
- 02 Большие формы – uncontrolled.
- 03 Файлы – всегда uncontrolled.

ПОДВОДНЫЕ КАМНИ

- 01 Controlled + большая форма – лагает.
- 02 Uncontrolled + динамика – нужно onChange.
- 03 value=undefined → React переключит в uncontrolled (warning).

КОД

// Сравнение

```
// controlled
<input value={v} onChange={e => setV(e.target.value)} />
// uncontrolled
<input defaultValue={v} ref={ref} />
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем react-hook-form Controller помогает?
- 02 Что такое flip controlled → uncontrolled?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/react-dom/components/input>

Когда использовать React Hook Form?

ОПИСАНИЕ

RHF полезен на формах от 5+ полей: subscription-based архитектура минимизирует re-рендеры, есть встроенные resolver для Zod/Yup, удобный API (register, handleSubmit). На простой форме из 1-2 полей overhead зависимости – лишний.

КАК РАБОТАЕТ

- 01 useForm() → register, handleSubmit, formState.
- 02 register('name') → spread в input.
- 03 zodResolver(schema) – для Zod-валидации.
- 04 Subscription per field – минимальный re-render.
- 05 Controller – для controlled-обёрток (date pickers).

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любая форма от 5 полей.
- 02 Multi-step forms.
- 03 Динамические поля (useFieldArray).
- 04 Перфоманс-критичные формы.

ПОДВОДНЫЕ КАМНИ

- 01 Простая форма из 1-2 полей – overkill.
- 02 Без resolver валидация ad-hoc и плохо типизирована.
- 03 Custom-input с onChange/onBlur требуют Controller.

КОД

// RHF + Zod

```
const { register, handleSubmit, formState: { errors } } =  
  useForm<V>({ resolver: zodResolver(Schema) });  
<input {...register('email')} />
```

МОГУТ ДОСПРОСИТЬ

- 01 Что такое useFieldArray?
- 02 Чем Formik отличается от RHF?

ПОЛНАЯ СТАТЬЯ

<https://react-hook-form.com/>

ВОПРОС Q04

Когда форма без библиотеки лучше?

ОПИСАНИЕ

Маленькие формы (1-3 поля) – useState на каждое поле + ручной submit. Это меньше зависимостей, меньше абстракций, проще читать. Также для one-off форм с уникальной логикой сторонняя библиотека добавляет больше, чем экономит.

КАК РАБОТАЕТ

- 01 useState per field или один объект.
- 02 onSubmit: e.preventDefault() + сбор данных.
- 03 Базовая валидация через if-проверки.
- 04 Native HTML5 validation как первая линия (required, pattern).

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Login form (email + password).
- 02 Search bar.
- 03 Single-field input (filter, comment).
- 04 Прототипы и MVP.

ПОДВОДНЫЕ КАМНИ

- 01 Когда форма растёт – пора на RHF.
- 02 Без валидации – user-баги.
- 03 Каждый рендер → setState → re-render всей формы.

КОД

// Простая форма

```
const [email, setEmail] = useState('');
const onSubmit = (e) => { e.preventDefault(); api.login(email); };
```

МОГУТ ДОСПРОСИТЬ

- 01 Когда мигрировать на RHF?
- 02 Что даёт HTML5 validation?

ПОЛНАЯ СТАТЬЯ

<https://developer.mozilla.org/en-US/docs/Learn/Forms>

ВАЛИДАЦИЯ**CLIENT / SERVER / СИНХРОНИЗАЦИЯ****ЧТО УЧИМ В ЭТОТ ДЕНЬ**

Где валидировать и как показывать ошибки.

ВОПРОСЫ ДНЯ

- Q05 Как валидировать на client?
- Q06 Как валидировать на server?
- Q07 Как синхронизировать client и server validation?
- Q08 Как показывать ошибки доступно?

ВОПРОС Q05

Как валидировать на client?

ОПИСАНИЕ

Schema-first через Zod / Valibot / Yup: одна схема даёт и тип, и валидацию. Поля валидируются на blur (показать ошибку), submit (полная валидация), иногда onChange (для критичных). Не дублировать с сервером – синхронизация (см. Q7).

КАК РАБОТАЕТ

- 01 Zod-схема + zodResolver(RHF).
- 02 Валидация on blur по умолчанию.
- 03 errors из formState.
- 04 FieldError → показывать рядом с полем + aria-invalid.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Форматы (email, телефон, URL).
- 02 Длина, диапазон чисел.
- 03 Сложные правила через .refine().

ПОДВОДНЫЕ КАМНИ

- 01 Только client → security-уязвимости. Сервер обязан.
- 02 Валидация на каждый keystroke – дёргает пользователя.
- 03 Дубль валидации client/server – расходится.

КОД

// Zod schema

```
const Schema = z.object({
  email: z.string().email(),
  age: z.number().int().min(18).max(120),
});
```

МОГУТ ДОСПРОСИТЬ

- 01 Что такое .refine() в Zod?
- 02 Как валидировать пароль (custom rules)?

ПОЛНАЯ СТАТЬЯ

<https://zod.dev/>

Как валидировать на server?

ОПИСАНИЕ

Сервер обязан валидировать ВСЕ входы — он не доверяет клиенту. Часто та же Zod-схема, переиспользуемая через shared package в monorepo. Возвращать структурированные ошибки: общие + field-level, чтобы клиент мог показать рядом с полем.

КАК РАБОТАЕТ

- 01 Schema.parse(req.body) → throws / Result.
- 02 Сервер возвращает 400 Bad Request с массивом ошибок.
- 03 Структура ошибок: { field, message, code }.
- 04 Schema shared между FE и BE (monorepo / npm package).

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любой POST/PUT/PATCH.
- 02 API GraphQL — schema-validation встроена.
- 03 Перевалидация uploads после client-side.

ПОДВОДНЫЕ КАМНИ

- 01 Только client-side — security hole.
- 02 Несогласованные сообщения client/server.
- 03 Server возвращает text вместо структуры — клиент не парсит.

КОД

// Express + Zod

```
app.post('/users', (req, res) => {
  const r = UserSchema.safeParse(req.body);
  if (!r.success) return res.status(400).json(r.error.issues);
  // ...
});
```

МОГУТ ДОСПРОСИТЬ

- 01 Какой формат ошибок выбрать?
- 02 Как переиспользовать schema FE↔BE?

ПОЛНАЯ СТАТЬЯ

<https://zod.dev/>

ВОПРОС Q07

Как синхронизировать client и server validation?

ОПИСАНИЕ

Лучший способ – shared schema в monorepo: одна Zod-схема импортируется и FE, и BE. Если monorepo нет – генерация типов из OpenAPI/JSON Schema. Структура ошибок единая. Server-ошибки мапятся в form-state клиента (setError по полю).

КАК РАБОТАЕТ

- 01 Monorepo + shared/types/zod-schemas.
- 02 Codegen из OpenAPI (openapi-typescript).
- 03 Server-error → setError('field', { message }).
- 04 FE первым уровнем фильтрует, server подтверждает.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любой fullstack TS-проект.
- 02 B2B SaaS с консистентным контрактом.

ПОДВОДНЫЕ КАМНИ

- 01 Расхождение → пользователь видит «Email valid» на FE, ошибку на BE.
- 02 Server-only-правила (uniqueness email) FE не покрывает.
- 03 Без типов ошибок – ad-hoc парсинг.

КОД

// Мappings server errors

```
try { await api.submit(values); }
catch (e) {
  if (e.fieldErrors) Object.entries(e.fieldErrors).forEach(([k, v]) =>
    setError(k, { message: v }));
}
```

МОГУТ ДОСПРОСИТЬ

- 01 Что такое contract testing?
- 02 Чем tRPC упрощает?

ПОЛНАЯ СТАТЬЯ

<https://trpc.io/docs>

Как показывать ошибки доступно?

ОПИСАНИЕ

Каждое поле имеет `aria-invalid` + `aria-describedby`, ссылающееся на сообщение об ошибке. Сообщение в `<p id="...-error">`. Screen reader озвучивает ошибку при focus. Группа ошибок формы – `role="alert"` или `live-region`. Цвет недостаточен – нужен иконка + текст.

КАК РАБОТАЕТ

- 01 `<input aria-invalid={!error} aria-describedby={` ${id}-error`} />`.
- 02 `<p id={` ${id}-error`} role="alert">{error.message}</p>`.
- 03 Цвет (красный) + иконка + текст – не только цвет.
- 04 Focus first invalid field после submit.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любая форма.
- 02 Особенно – финансовые / медицинские.
- 03 B2B – где есть keyboard-only пользователи.

ПОДВОДНЫЕ КАМНИ

- 01 Только цвет – `colorblind` не видят.
- 02 Без `aria` – screen reader не озвучивает.
- 03 `Tooltip-only error` – не доступен с клавиатуры.

КОД

// Доступная ошибка

```
<input id="email" aria-invalid={!err}
  aria-describedby={err ? 'email-err' : undefined} />
{err && <p id="email-err" role="alert">{err.message}</p>}
```

МОГУТ ДОСПРОСИТЬ

- 01 Что такое `aria-live`?
- 02 Как сделать summary всех ошибок доступным?

ПОЛНАЯ СТАТЬЯ

<https://www.w3.org/WAI/tutorials/forms/notifications/>

ДЕНЬ 3 НАГРУЗКА СРЕДНИЙ 3 ВОПРОСА

TOUCHED / DIRTY ASYNC VALIDATION STALE**ЧТО УЧИМ В ЭТОТ ДЕНЬ**

Управление состоянием полей и async-проверками.

ВОПРОСЫ ДНЯ

- Q09 Как управлять touched/dirty state?
- Q10 Как проектировать async validation?
- Q11 Как отменять stale validation request?

Как управлять touched/dirty state?

ОПИСАНИЕ

touched – поле получало фокус и потеряло (blur). dirty – значение изменилось от initial. RHF трекает оба в formState. Используется для: показать ошибку только после blur (не дёргать пользователя), включить submit-кнопку только если форма dirty, показать prompt при leave с несохранёнными изменениями.

КАК РАБОТАЕТ

- 01 RHF: formState.touchedFields, dirtyFields, isDirty.
- 02 Mode: 'onBlur' / 'onChange' / 'onSubmit' / 'all' – когда валидировать.
- 03 isDirty – есть ли изменения от defaultValues.
- 04 Dirty-prompt при unmount / route change.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 UX-дружественная валидация (показать после blur).
- 02 Disabled submit пока не dirty.
- 03 Подтверждение «уйти без сохранения?».
- 04 Сохранять draft только при dirty.

ПОДВОДНЫЕ КАМНИ

- 01 Mode 'onChange' – валидация на каждый keystroke, дёргает.
- 02 isDirty не учитывает «вернулось к initial» – туда-обратно даёт dirty.
- 03 Reset формы → touchedFields очищаются.

КОД

// Использование

```
const { formState: { touchedFields, isDirty, errors } } = useForm({ mode: 'onBlur' });
{touchedFields.email && errors.email && <p>{errors.email.message}</p>}
```

МОГУТ ДОСПРОСИТЬ

- 01 Что такое RHF mode?
- 02 Как реализовать prompt-on-leave?

ПОЛНАЯ СТАТЬЯ

<https://react-hook-form.com/docs/useform/formstate>

ВОПРОС Q10

Как проектировать async validation?

ОПИСАНИЕ

Async validation – проверка на сервере (uniqueness, available username). Реализуется через async-resolver или через .refine() в Zod с async-функцией. Запускается после debounce или на blur. Защищаться от race condition (см. Q11) и показывать loading-state у поля.

КАК РАБОТАЕТ

- 01 Zod: `z.string().refine(async (v) => api.checkUnique(v))`.
- 02 RHF + `asyncResolver`.
- 03 Debounce 300-500мс перед запросом.
- 04 Loading-state у поля (`spinner` / `opacity`).
- 05 `AbortController` для отмены устаревших запросов.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Username / email на регистрации.
- 02 Промокод (валиден ли).
- 03 Адрес доставки (поддерживается ли).
- 04 Серверные правила, недоступные FE.

ПОДВОДНЫЕ КАМНИ

- 01 Без debounce – запросы на каждое нажатие.
- 02 Race condition – старая ошибка перезагнала валидную форму.
- 03 Server timeout → форма "висит".
- 04 Спам валидации блокирует submit.

КОД

```
// Async refine
const Schema = z.object({
  username: z.string().min(3).refine(
    async (v) => (await api.checkUnique(v)).ok,
    { message: 'Уже занято' }
  ),
});
```

МОГУТ ДОСПРОСИТЬ

- 01 Как комбинировать sync + async?
- 02 Чем `.refine()` от `.superRefine()` отличается?

ПОЛНАЯ СТАТЬЯ

<https://zod.dev/?id=refine>

ВОПРОС Q11

Как отменять stale validation request?

ОПИСАНИЕ

Старая async-валидация может вернуться позже новой и перезагереть актуальное состояние. Защита: AbortController перед каждым новым запросом, или проверка по version-id. RHF сам защищается от перезаписи через internal-counter, но fetch отменять всё равно нужно для производительности.

КАК РАБОТАЕТ

- 01 AbortController на запрос.
- 02 При новом значении — abort предыдущего.
- 03 В onSettled очистить state валидации.
- 04 Throttle / debounce + abort = идеальная пара.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любая async-валидация.
- 02 Search-with-validate (например, 'свободен ли username').

ПОДВОДНЫЕ КАМНИ

- 01 Без abort — N запросов в полёте.
- 02 AbortError путают с сетевой ошибкой → ложное «error».
- 03 Race на быстрой печати.

КОД

// Abort стейлой валидации

```
let ctrl: AbortController | null = null;  
async function validate(v: string) {  
  ctrl?.abort();  
  ctrl = new AbortController();  
  return api.check(v, { signal: ctrl.signal });  
}
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем version-counter отличается от AbortController?
- 02 Как отличать AbortError от сетевой?

ПОЛНАЯ СТАТЬЯ

<https://developer.mozilla.org/en-US/docs/Web/API/AbortController>

MULTI-STEP

DRAFTS

AUTOSAVE

ЧТО УЧИМ В ЭТОТ ДЕНЬ

Длинные wizards и сохранение состояния.

ВОПРОСЫ ДНЯ

Q12 Как проектировать multi-step form?

Q13 Как сохранять draft?

Q14 Как обрабатывать autosave?

ВОПРОС Q12

Как проектировать multi-step form?

ОПИСАНИЕ

Wizard – multi-step с явными шагами, навигацией, валидацией каждого шага, state между шагами. Архитектура – DU состояний или XState, контейнер-логики, step-components. Сохранять progress в URL для shareable-ссылки и draft в storage.

КАК РАБОТАЕТ

- 01 Шаги как DU: { step: 'address' } | { step: 'payment' }.
- 02 Контейнер хранит общий state.
- 03 Каждый шаг получает state + onNext + onBack.
- 04 Валидация шага через Zod (только нужные поля).
- 05 URL ?step=payment для возврата.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Onboarding.
- 02 Checkout-флоу.
- 03 Многошаговая регистрация.
- 04 Мастера настройки.

ПОДВОДНЫЕ КАМНИ

- 01 Один большой объект state без структуры – путаница.
- 02 Шаги жёстко зашиты – нельзя переставить.
- 03 Refresh теряет state – нужен localStorage / URL.
- 04 Backend-флоу не синхронизирован с frontend.

КОД

// DU состояний

```
type Wizard =  
  | { step: 'info'; data: Partial<UserInfo> }  
  | { step: 'address'; data: UserInfo & Partial<Address> }  
  | { step: 'review'; data: FullData };
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем XState помогает?
- 02 Как сохранять draft между сеансами?

ПОЛНАЯ СТАТЬЯ

<https://statelily.ai/docs>

Как сохранять draft?

ОПИСАНИЕ

Draft – промежуточное состояние формы, переживающее reload. Хранится в localStorage / sessionStorage / IndexedDB / на сервере. При loadе – восстановить значения. Чистить после успешного submit. Учитывать TTL и privacy.

КАК РАБОТАЕТ

- 01 Watch values → debounced save → storage.
- 02 On mount → load из storage → reset(values).
- 03 On successful submit → cleanup.
- 04 TTL (например, 7 дней) для очистки старых.
- 05 Server-side draft для cross-device.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Длинные формы (резюме, заявка на ипотеку).
- 02 Multi-step wizards.
- 03 User-generated content (post drafts).

ПОДВОДНЫЕ КАМНИ

- 01 Sensitive-данные в localStorage – security.
- 02 Draft не подходит между разными пользователями (logout).
- 03 Storage quota.
- 04 Конфликт draft-версий, если открыли в двух вкладках.

КОД

// Debounced autosave

```
const watched = watch();
useEffect(() => {
  const t = setTimeout(() => localStorage.setItem(KEY, JSON.stringify(watched)), 500);
  return () => clearTimeout(t);
}, [watched]);
```

МОГУТ ДОСПРОСИТЬ

- 01 Где хранить sensitive draft?
- 02 Что делать с конфликтом версий?

ПОЛНАЯ СТАТЬЯ

https://developer.mozilla.org/en-US/docs/Web/API/Web_Storage_API

ВОПРОС Q14

Как обрабатывать autosave?

ОПИСАНИЕ

Autosave = save без явного клика. Должен быть незаметным: debounce (500-2000мс), показывать статус «Saving.../Saved», обрабатывать сетевые ошибки (retry с backoff), не перезаписывать optimistic-данные более старым ответом, конфликты разрешать через version или last-write-wins.

КАК РАБОТАЕТ

- 01 Watch + debounce 500-2000мс.
- 02 useMutation с retry.
- 03 Status-indicator: idle/saving/saved/error.
- 04 Conflict resolution: ETag / version-counter.
- 05 Optimistic UI + rollback при ошибке.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Editor (Notion, Google Docs).
- 02 Settings page.
- 03 Long forms (CV builder).

ПОДВОДНЫЕ КАМНИ

- 01 Слишком частый save – нагрузка на BE.
- 02 Без status-indicator пользователь не уверен.
- 03 Race с manual-submit.
- 04 Conflict → данные теряются.

КОД

// Autosave с indicator

```
const m = useMutation({ mutationFn: api.save });
const status = m.isPending ? 'saving' : m.isError ? 'error' : 'saved';
<div aria-live="polite">{status}</div>
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем optimistic save отличается от обычного?
- 02 Как разрешать конфликты?

ПОЛНАЯ СТАТЬЯ

<https://tanstack.com/query/latest/docs/framework/react/guides/optimistic-updates>

OPTIMISTIC SUBMIT**DOUBLE SUBMIT****DISABLED****ЧТО УЧИМ В ЭТОТ ДЕНЬ**

Защита от дублей и UX отправки.

ВОПРОСЫ ДНЯ

Q15 Как делать optimistic form submit?

Q16 Как защищаться от double submit?

Q17 Как обрабатывать disabled submit?

ВОПРОС Q15

Как делать optimistic form submit?

ОПИСАНИЕ

Optimistic submit – UI обновляется до получения ответа сервера. При error – rollback. Реализуется через useMutation onMutate / onError / onSettled. UX-выгода: мгновенная реакция, не нужно ждать сети. Особенно ценно на медленных сетях / mobile.

КАК РАБОТАЕТ

- 01 onMutate: snapshot текущего → apply optimistic → return ctx.
- 02 onError: rollback из ctx.
- 03 onSettled: invalidate / refetch.
- 04 UI отображает локально в момент submit.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Like / favorite / bookmark.
- 02 Add to cart.
- 03 Quick edits (inline).
- 04 Toggle / checkbox в списке.

ПОДВОДНЫЕ КАМНИ

- 01 Без rollback – обманывает пользователя.
- 02 Сложно для новых entity (нет ID до серверного ответа).
- 03 Race с другими mutations.

КОД

```
// Optimistic mutation
```

```
useMutation({
  mutationFn: api.update,
  onMutate: async (next) => {
    const prev = qc.getQueryData(KEY);
    qc.setQueryData(KEY, next);
    return { prev };
  },
  onError: (_e, _v, ctx) => qc.setQueryData(KEY, ctx?.prev),
  onSettled: () => qc.invalidateQueries({ queryKey: KEY }),
});
```

МОГУТ ДОСПРОСИТЬ

- 01 Как сделать optimistic для list-add?
- 02 Что показать при rollback?

ПОЛНАЯ СТАТЬЯ

<https://tanstack.com/query/latest/docs/framework/react/guides/optimistic-updates>

ВОПРОС Q16

Как защищаться от double submit?

ОПИСАНИЕ

Двойной клик / двойная отправка → дублирующиеся записи. Защита: disable submit button во время pending, idempotency-key в HTTP-запросе, защита на сервере (unique constraint), loading-state на форме. RHF умеет isSubmitting автоматически.

КАК РАБОТАЕТ

- 01 isSubmitting из formState – disable button.
- 02 Idempotency-Key header в POST.
- 03 Server: unique constraint + 409 Conflict.
- 04 Локальный флаг + setTimeout cleanup.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Order submit.
- 02 Payment / transfer.
- 03 Create-resource эндпоинты.
- 04 Любая mutation с побочными эффектами.

ПОДВОДНЫЕ КАМНИ

- 01 Только client-disable – пользователь refreshes страницу.
- 02 Без idempotency на BE – race создаёт дубль.
- 03 Disabled выглядит сломанным без spinner.

КОД

```
// RHF + disable
```

```
const { handleSubmit, formState: { isSubmitting } } = useForm();  
<button disabled={isSubmitting}>{isSubmitting ? 'Saving...' : 'Save'}</button>
```

МОГУТ ДОСПРОСИТЬ

- 01 Что такое Idempotency-Key?
- 02 Как сервер обрабатывает дубль?

ПОЛНАЯ СТАТЬЯ

https://stripe.com/docs/api/idempotent_requests

Как обрабатывать disabled submit?

ОПИСАНИЕ

Disabled submit – кнопка отключена пока форма невалидна / в submit. UX-нюансы: причина disabled должна быть видна (tooltip / validation-summary), aria-disabled vs disabled (aria-disabled доступен focus и tooltip), показать ошибки при попытке submit на disabled.

КАК РАБОТАЕТ

- 01 isSubmitting → disabled.
- 02 Form невалидна – disable submit.
- 03 aria-disabled="true" – доступен focus, видно tooltip с причиной.
- 04 Validation summary над формой.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Сложные формы с обязательными полями.
- 02 Payment форма (валидный card необходим).
- 03 Submit с условиями (термины приняты).

ПОДВОДНЫЕ КАМНИ

- 01 disabled без подсказки – пользователь не понимает почему.
- 02 Skip native disabled-state для accessible – использовать aria-disabled.
- 03 Disabled prevents focus → нельзя переместить курсор.

КОД

// aria-disabled

```
<button aria-disabled={!isValid} onClick={(e) => {  
  if (!isValid) { e.preventDefault(); showErrors(); }  
}}>Submit</button>
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем disabled от aria-disabled отличается?
- 02 Как показать причину disabled?

ПОЛНАЯ СТАТЬЯ

<https://www.w3.org/TR/wai-aria-1.2/#aria-disabled>

ДЕНЬ 6

НАГРУЗКА

ТЯЖЁЛЫЙ

2 ВОПРОСА

FILE UPLOAD

LARGE FORM PERFORMANCE

ЧТО УЧИМ В ЭТОТ ДЕНЬ

Файлы и формы с сотней полей.

ВОПРОСЫ ДНЯ

Q18 Как проектировать file upload?

Q19 Как проектировать large form performance?

Как проектировать file upload?

ОПИСАНИЕ

File upload: HTML `<input type="file">` (всегда uncontrolled), drag-n-drop с `FileList`, progress-индикатор через `XMLHttpRequest` или `fetch` с `readable-streams`. Большие файлы – chunked upload (S3 multipart, tus). Превью изображений – `URL.createObjectURL`.

Валидация: тип, размер.

КАК РАБОТАЕТ

- 01 `<input type="file" accept="image/*" multiple />`.
- 02 Drag-n-drop: `ondragover.preventDefault + ondrop → e.dataTransfer.files`.
- 03 `FormData + fetch / XHR` (XHR для progress).
- 04 Multipart upload для больших файлов.
- 05 `URL.createObjectURL` для превью.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Загрузка аватара.
- 02 Импорт CSV / Excel.
- 03 Attachments в чате / форме.
- 04 Загрузка медиа.

ПОДВОДНЫЕ КАМНИ

- 01 `fetch` не показывает progress download (есть Streams).
- 02 Без `URL.revokeObjectURL` – утечка памяти.
- 03 Не валидируем тип на сервере → security.
- 04 Большие файлы без chunking – таймаут / out-of-memory.

КОД

// Upload + progress (XHR)

```
function upload(file: File, onProgress: (n: number) => void) {
  return new Promise((resolve, reject) => {
    const xhr = new XMLHttpRequest();
    xhr.upload.onprogress = (e) => onProgress(e.loaded / e.total);
    xhr.onload = () => resolve(xhr.response);
    xhr.onerror = reject;
    const fd = new FormData();
    fd.append('file', file);
    xhr.open('POST', '/upload');
    xhr.send(fd);
  });
}
```

МОГУТ ДОСПРОСИТЬ

- 01 Что такое resumable upload?
- 02 Как валидировать файл до upload?

ПОЛНАЯ СТАТЬЯ

<https://developer.mozilla.org/en-US/docs/Web/API/File>

Как проектировать large form performance?

ОПИСАНИЕ

Большие формы (50+ полей) лагают при controlled-подходе. Решения: react-hook-form (subscription per field, минимальные re-renders), разбить на секции с lazy-load, мемо полей (React.мемо + стабильные props), virtualization для очень длинных списков полей.

КАК РАБОТАЕТ

- 01 RHF – subscribe per field.
- 02 Field-component через React.мемо.
- 03 useFieldArray для динамических списков.
- 04 Lazy-section через React.lazy + Suspense.
- 05 Виртуализация для 100+ строк (react-window).

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Конструкторы (form builder).
- 02 Annual reports / tax forms.
- 03 Bulk-edit таблиц.
- 04 B2B-админки с большими формами.

ПОДВОДНЫЕ КАМНИ

- 01 Watch() на верхнем уровне – каждый change ререндерит всё.
- 02 Inline onChange = нестабильная функция = мемо не работает.
- 03 Слишком много полей в одном экране – UX-проблема, не только perf.

КОД

```
// Field-memo
const FormField = React.memo(({ name, register }) => (
  <input {...register(name)} />
));
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем useFieldArray помогает?
- 02 Как избежать watch-cascade?

ПОЛНАЯ СТАТЬЯ

<https://react-hook-form.com/docs/usefieldarray>

ДЕНЬ 7 НАГРУЗКА СРЕДНИЙ 1 ВОПРОС

ТЕСТИРОВАНИЕ COMPLEX FORM

ЧТО УЧИМ В ЭТОТ ДЕНЬ

Как покрыть форму тестами.

ВОПРОСЫ ДНЯ

Q20 Как тестировать complex form?

Как тестировать complex form?

ОПИСАНИЕ

Тестирование уровня unit (валидация Zod-схемы, helpers), integration (форма + поля + submit) через testing-library, e2e (полный flow) через Playwright. Принципы: тестируем поведение, не implementation. Использовать getByRole / getByLabelText вместо querySelector. Mock API через MSW.

КАК РАБОТАЕТ

- 01 Schema-юнит: Schema.parse(input) → expect throws.
- 02 Integration: render(<Form />), userEvent.type, click Submit.
- 03 Mock API через MSW.
- 04 E2E: Playwright + Page Object.
- 05 Visual regression – Chromatic / Percy.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любая критичная форма (checkout, signup).
- 02 Forms с conditional fields.
- 03 Multi-step wizards.
- 04 Accessibility-чек.

ПОДВОДНЫЕ КАМНИ

- 01 Тесты на implementation (data-testid везде) – хрупкие.
- 02 Без MSW – flaky зависят от сети.
- 03 Без ally-проверки – забывают.
- 04 E2E на каждый сценарий – медленно.

КОД

```
// Integration test
```

```
test('submit valid form', async () => {
  render(<SignupForm onSubmit={mock} />);
  await userEvent.type(screen.getByLabelText(/email/i), 'a@a.com');
  await userEvent.click(screen.getByRole('button', { name: /submit/i }));
  expect(mock).toHaveBeenCalledWith({ email: 'a@a.com' });
});
```

МОГУТ ДОСПРОСИТЬ

- 01 Что такое MSW?
- 02 Чем getByRole от getByTestId лучше?

ПОЛНАЯ СТАТЬЯ

<https://testing-library.com/docs/queries/about/#priority>

СДАЛ ЛИ ЗАЧЁТ

ПРОЙДИ ВСЛУХ БЕЗ ПОДГЛЯДЫВАНИЙ ЕСЛИ ЗАСТРЯЛ – ВЕРНИСЬ В НУЖНЫЙ БИЛЕТ

- ☐ Расписать архитектуру сложной формы (Q1)
- ☐ Выбрать controlled/uncontrolled и сказать когда RHF (Q2, Q3, Q4)
- ☐ Спроектировать client + server + sync валидацию (Q5, Q6, Q7)
- ☐ Сделать ошибки доступными и описать touched/dirty (Q8, Q9)
- ☐ Спроектировать async-валидацию и отмену стейлов (Q10, Q11)
- ☐ Спроектировать multi-step + draft + autosave (Q12, Q13, Q14)
- ☐ Описать optimistic submit + double-submit + disabled (Q15, Q16, Q17)
- ☐ Спроектировать file upload и большие формы (Q18, Q19)
- ☐ Описать стратегию тестирования формы (Q20)